Name: Pratyush Kumar

Course: CSE 587, Deep Learning for Natural Language Processing

Semester: Spring 2026

Instructor: Dr. Rui Zang

Assignment 2

Problem (math_baseline):

(a) Script file - /scripts/evaluate_math_zero_shot.py

(b) Counts are as follows:

   a. 24 examples with format_reward = 1 and answer_reward = 1

   b. 34 examples with format_reward = 1 and answer_reward = 0

   c. 1442 examples with format_reward = 0 and answer_reward = 0

(c) On the MATH validation set, the zero-shot Qwen/Qwen2.5-Math-1.5B baseline achieved answer_reward = 0.0160, format_reward = 0.0387, and reward = 0.0160. This is a very weak performance: only 24/1500 examples received both the format and answer rewards 1, and most generations failed to produce a valid formatted final answer.

Problem (tokenize_prompt_and_output):

Method implemented in alignment/sft.py

```
def tokenize_prompt_and_output(
    prompt_strs: list[str],
    output_strs: list[str],
    tokenizer: PreTrainedTokenizerBase,
) -> dict[str, Tensor]:
```

Problem (compute_entropy):

Method implemented in alignment/sft.py

```
def compute_entropy(logits: Tensor) -> Tensor:
```

Problem (get_response_log_probs):

Method implemented in alignment/sft.py

```
def get_response_log_probs(
    model: PreTrainedModel,
    input_ids: Tensor,
    labels: Tensor,
    return_token_entropy: bool = False,
) -> dict[str, Tensor]:
```

Problem (masked_normalize):

Method implemented in alignment/sft.py

```
def masked_normalize(
    tensor: torch.Tensor,
    mask: torch.Tensor,
    normalize_constant: float,
    dim: int | None = None,
) -> torch.Tensor:
```

Problem (sft_microbatch_train_step):

Method implemented in alignment/sft.py

```
def sft_microbatch_train_step(
    policy_log_probs: torch.Tensor,
    response_mask: torch.Tensor,
    gradient_accumulation_steps: int,
    normalize_constant: float = 1.0,
) -> tuple[torch.Tensor, dict[str, torch.Tensor]]:
```

Problem (log_generations):

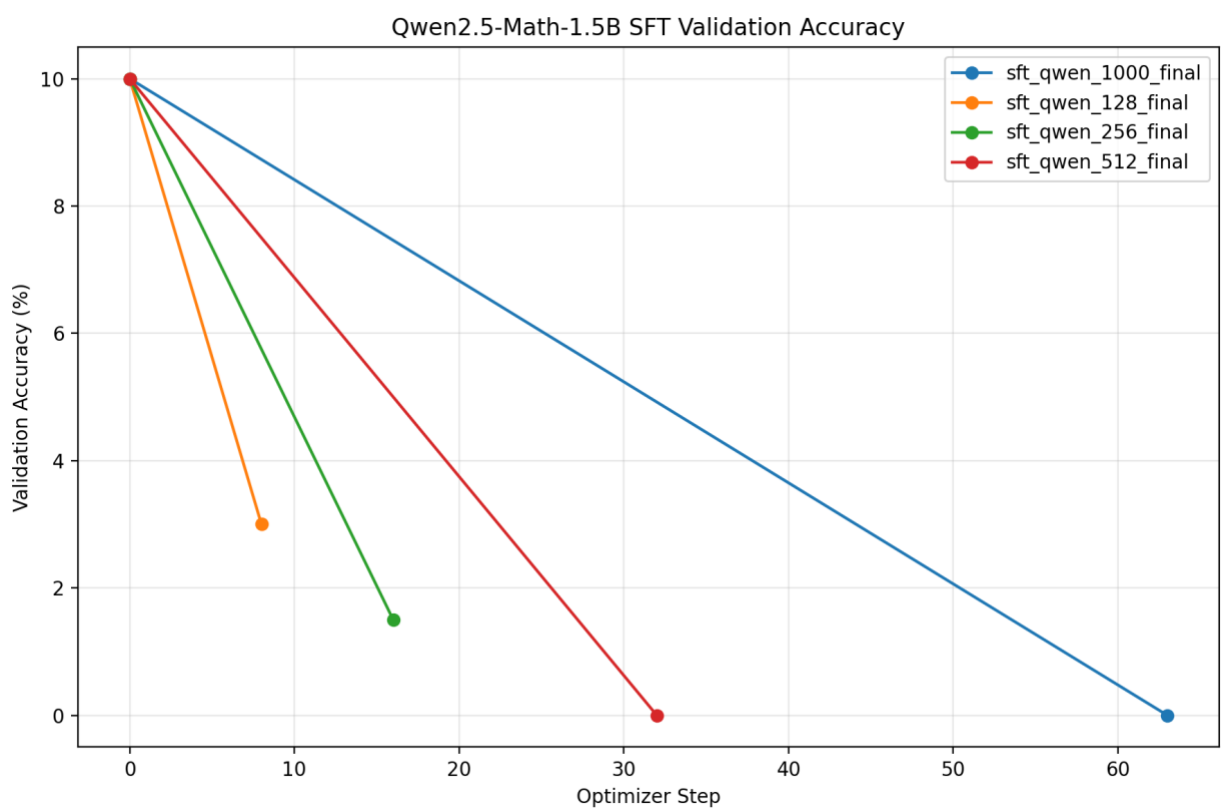
Method implemented in alignment/logging_utils.py

```
def log_generations(
    prompts: Sequence[str],
    generations: Sequence[str],
    *,
    ground_truths: Sequence[str | None] | None = None,
    references: Sequence[str | None] | None = None,
    reward_info: Sequence[Mapping[str, Any] | float | int | None] | None =
None,
    rewards: Sequence[float | int | None] | None = None,
    avg_token_entropies: Sequence[float | None] | None = None,
    response_lengths: Sequence[int | None] | None = None,
    is_correct: Sequence[bool | None] | None = None,
    metadata: Sequence[Mapping[str, Any] | None] | None = None,
    output_path: str | Path | None = None,
    wandb_run: Any | None = None,
    table_name: str = "generations",
    summary_name: str = "generation_summary",
```

```
    step: int | None = None,
    max_rows: int | None = None,
) -> GenerationLog:
```

Problem (sft_experiment):

(a) I trained Qwen/Qwen2.5-Math-1.5B with SFT on 128, 256, 512, and 1000 examples from data/sft.jsonl. Using a learning rate of 2e-5, a per-device batch size of 1, gradient accumulation of 16, and one epoch, validation accuracy decreased after training for all dataset sizes. The base model started at about 10% validation accuracy on the 200-example validation subset, but the final accuracies dropped to 3.0%, 1.5%, 0.0%, and 0.0% for 128, 256, 512, and 1000 examples, respectively. Thus, under this configuration, increasing the number of SFT examples did not improve performance and instead degraded it.



PS: This plot uses 200 validation examples, not the full 1500-example validation set (due to computation constraints).

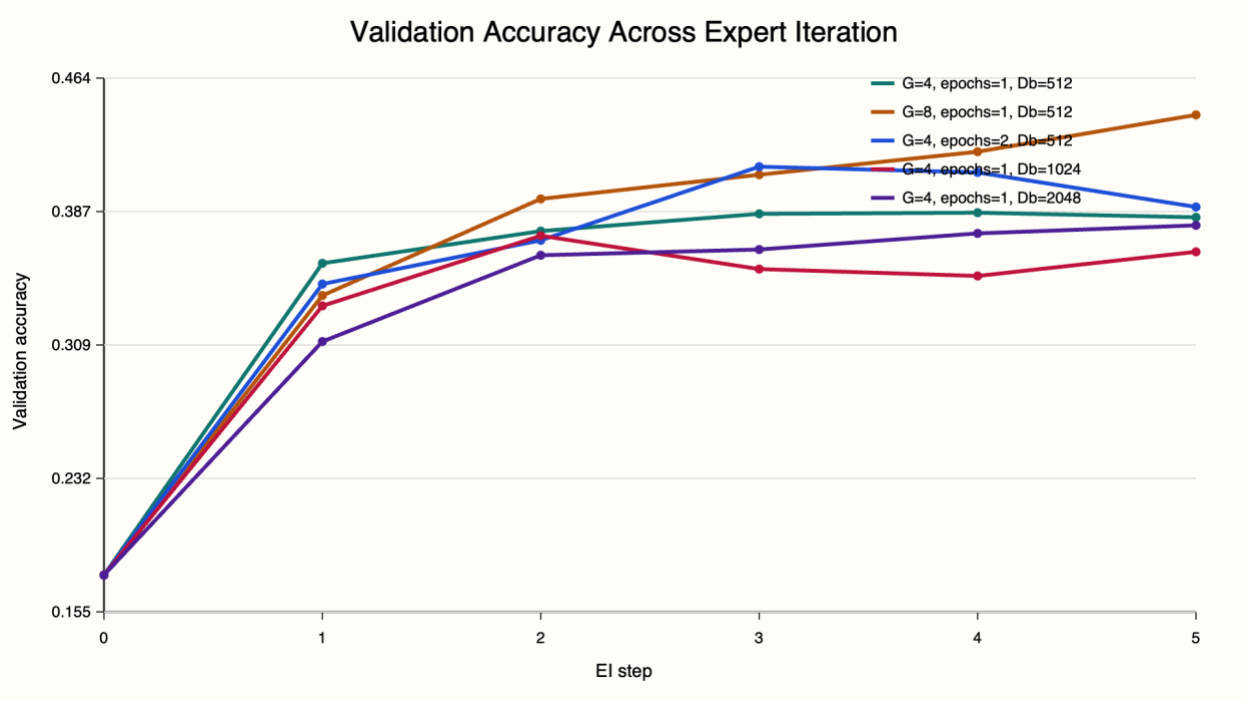
(b) Filtering the reasoning SFT dataset to keep only examples that produced the correct answer reduced the training set from 1000 examples to 631 examples, so 63.1% of the data was retained. I then reran SFT on the full filtered dataset and compared it

with the previous unfiltered SFT experiment using the same lightweight training configuration. The validation accuracy curves were essentially unchanged: both the unfiltered and filtered runs achieved a best validation accuracy of 6% on the 50-example validation subset. In both runs, the best accuracy occurred at step 0, indicating that training did not improve over the initial checkpoint. Overall, filtering removed many incorrect reasoning traces but did not improve validation accuracy in this experiment.

<u>Problem (expert_iteration_experiment)</u>:

<u>Deliverable 1</u>

Validation accuracy curves for expert iteration:



I evaluated multiple configurations that vary both rollout count and SFT epochs, including G=4, epochs=1, G=8, epochs=1, and G=4, epochs=2, along with additional Db comparisons at 512, 1024, and 2048.

<u>Deliverable 2</u>

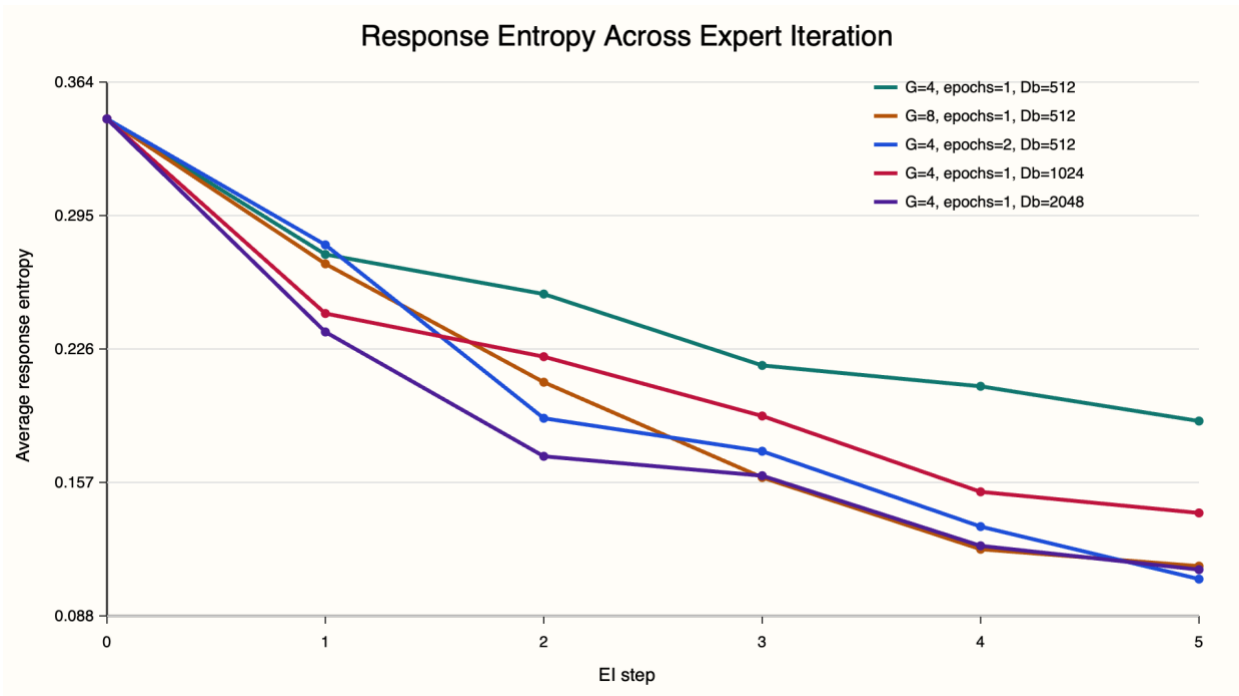
A model exceeding 15% validation accuracy was obtained easily. The best run was G=8, epochs=1, Db=512, which achieved 44.27% validation accuracy at EI step 5, with model checkpoint outputs/ei_math/g8_e1_db512/step_5/model.

Deliverable 3

Compared with my SFT baseline, which achieved only 10.0% validation accuracy, expert iteration improved performance dramatically, with all EI runs reaching at least 37.27% and the best run reaching 44.27%. Across EI steps, validation accuracy generally improved over the early EI iterations and then either continued increasing or plateaued, with larger rollout count helping most in this sweep, while increasing Db beyond 512 did not improve the best result.

Deliverable 4

The entropy-over-training plot is as follows



Problem (compute_group_normalized _rewards):

Function available in sft.py

```
def compute_group_normalized_rewards(
    reward_fn,
    rollout_responses: list[str],
    repeated_ground_truths: list[str],
    group_size: int,
    advantage_eps: float,
    normalize_by_std: bool,
) -> tuple[Tensor, Tensor, dict[str, float]]:
```

Problem (naïve_policy_gradient_loss):

The function definition is in alignment/sft.py

```
def compute_naive_policy_gradient_loss(
    raw_rewards_or_advantages: torch.Tensor,
    policy_log_probs: torch.Tensor,
) -> torch.Tensor:
    """Compute the per-token policy-gradient loss with broadcasted
rewards."""
    return -policy_log_probs * raw_rewards_or_advantages
```

Problem (compute_grpo_clip_loss):

The function definition is in alignment/sft.py

```
def compute_grpo_clip_loss(
    advantages: torch.Tensor,
    policy_log_probs: torch.Tensor,
    old_log_probs: torch.Tensor,
    cliprange: float,
) -> tuple[torch.Tensor, dict[str, torch.Tensor]]:
```

Problem (compute_policy_gradient_loss):

The function compute_policy_gradient_loss now dispatches by loss_type to:

- compute_naive_policy_gradient_loss(..., raw_rewards) for "no_baseline"
- compute_naive_policy_gradient_loss(..., advantages) for "reinforce_with_baseline"
- compute_grpo_clip_loss(...) for "grpo_clip"

It returns (loss, metadata) and raises ValueError when required arguments are missing, or loss_type is invalid. The function run_compute_policy_gradient_loss also delegates to that function.

Problem (masked_mean):

The function definition is in alignment/sft.py

```
def masked_mean(
    tensor: torch.Tensor,
    mask: torch.Tensor,
    dim: int | None = None,
) -> torch.Tensor:
```

Problem (grpo_microbatch_train_step):

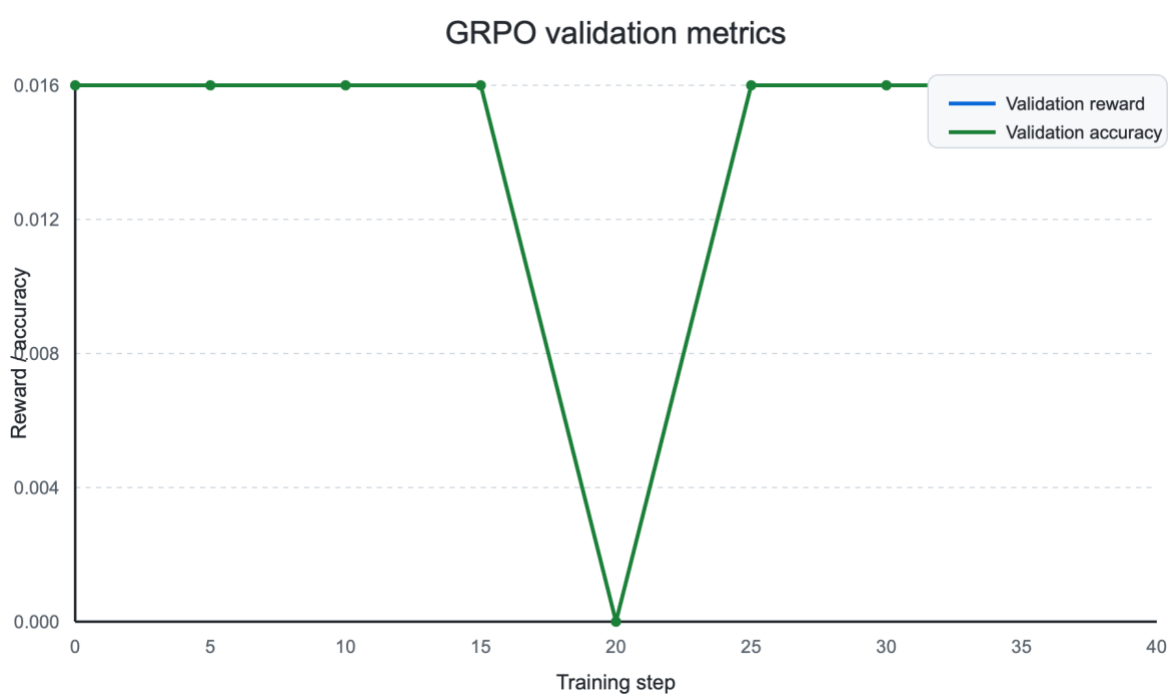
The function definition is in alignment/sft.py

```
def grpo_microbatch_train_step(
    policy_log_probs: torch.Tensor,
    response_mask: torch.Tensor,
    gradient_accumulation_steps: int,
    loss_type: Literal["no_baseline", "reinforce_with_baseline",
"grpo_clip"],
    raw_rewards: torch.Tensor | None = None,
    advantages: torch.Tensor | None = None,
    old_log_probs: torch.Tensor | None = None,
    cliprange: float | None = None,
) -> tuple[torch.Tensor, dict[str, torch.Tensor]]:
```

Problem (grpo_train_loop):

Implemented a full GRPO loop for MATH, logged validation rewards oversteps, and saved rollout samples over time.

On the available hardware and with the explored hyperparameters, the runs were either unstable or reward-sparse: some runs collapsed numerically, while the final stabilized run remained flat at roughly constant validation reward with little qualitative rollout change. The attached plots and rollout examples document these outcomes.



Validation accuracy and reward in comparison to the GRPO training stage on MATH. After safeguards were added, the run remained numerically stable, but the validation reward remained close to its starting value of roughly 0.016, with a transient decline at step 20 and no long-term improvement.

The same pattern was seen in the qualitative rollout inspection. The model frequently generated logical-looking mathematical explanations during the early, middle, and late validation rollouts, but many of the responses were shortened, improperly structured, or inaccurate. The rollouts did not collapse during the stabilized run, but they also did not significantly improve in accuracy over time. This aligns with the flat validation reward curve and implies that GRPO was insufficient to enhance MATH performance at this initial point and with these hyper-parameters.

The implementation was overall successful, but the experiment did not demonstrate improving validation rewards or progressively better rollouts on MATH.